

**LAPORAN PRAKTIKUM
PEMROGRAMAN MOBILE
MODUL 5**



CONNECT TO THE INTERNET

Oleh:

Abdurrahman Gilang Harjuna

NIM. 2310817110004

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
MEI 2025**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM PEMROGRAMAN I
MODUL 5

Laporan Praktikum Pemrograman Mobile Modul 5: Connect to the Internet List ini disusun sebagai syarat lulus mata kuliah Praktikum Pemrograman Mobile. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Abdurrahman Gilang Harjuna
NIM : 2310817110004

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Zulfa Auliya Akbar
NIM. 2210817210026

Muti`a Maulida S.Kom M.T.I
NIP. 19881027 201903 20 13

DAFTAR ISI

LEMBAR PENGESAHAN.....	2
DAFTAR ISI	3
DAFTAR GAMBAR.....	4
DAFTAR TABEL	5
SOAL 1.....	6
A. Source Code	6
B. Output Program.....	25
C. Pembahasan	26
D. Tautan Git	32

DAFTAR GAMBAR

Gambar 1. Screenshot hasil soal 1.....	25
Gambar 2. Screenshot hasil soal 1.....	26

DAFTAR TABEL

Table 1. Source Code Jawaban Soal 1.....	8
Table 2. Source Code Jawaban Soal 1.....	11
<i>Table 3. Source Code Jawaban Soal 1</i>	<i>13</i>
<i>Table 4. Source Code Jawaban Soal 1</i>	<i>13</i>
<i>Table 5. Source Code Jawaban Soal 1</i>	<i>14</i>
<i>Table 6. Source Code Jawaban Soal 1</i>	<i>14</i>
<i>Table 7. Source Code Jawaban Soal 1</i>	<i>15</i>
<i>Table 8. Source Code Jawaban Soal 1</i>	<i>17</i>
<i>Table 9. Source Code Jawaban Soal 1</i>	<i>17</i>
<i>Table 10. Source Code Jawaban Soal 1</i>	<i>18</i>
<i>Table 11. Source Code Jawaban Soal 1</i>	<i>19</i>
<i>Table 12. Source Code Jawaban Soal 1</i>	<i>22</i>
<i>Table 13. Source Code Jawaban Soal 1</i>	<i>24</i>

SOAL 1

1. Lanjutkan aplikasi Android yang sudah dibuat pada Modul 4 dengan menambahkan modifikasi sesuai ketentuan berikut:

- Gunakan networking library seperti Retrofit atau Ktor agar aplikasi dapat mengambil data dari remote API. Dalam penggunaan networking library, sertakan generic response untuk status dan error handling pada API dan Flow untuk data stream.
- Gunakan KotlinX Serialization sebagai library JSON.
- Gunakan library seperti Coil atau Glide untuk image loading.
- API yang digunakan pada modul ini bebas, contoh API gratis The Movie Database (TMDB) API yang menampilkan data film. Berikut link dokumentasi API: <https://developer.themoviedb.org/docs/getting-started>
- Implementasikan konsep data persistence (misalnya offline-first app, pengaturan dark/light mode, fitur favorite, dll)
- Gunakan caching strategy pada Room..
- Untuk Modul 5, bebas memilih UI yang ingin digunakan, antara berbasis XML atau Jetpack Compose.

Aplikasi harus mempertahankan fitur-fitur yang dibuat pada modul sebelumnya.

A. Source Code

1. HomeFragment.kt

```
1 package com.example.zennfit
2
3 import android.os.Bundle
4 import android.util.Log
5 import android.view.LayoutInflater
6 import android.view.View
7 import android.view.ViewGroup
8 import androidx.fragment.app.Fragment
9 import androidx.fragment.app.viewModels
10 import androidx.lifecycle.LifecycleScope
11 import androidx.recyclerview.widget.LinearLayoutManager
12 import com.example.zennfit.databinding.FragmentHomeBinding
13 import kotlinx.coroutines.flow.collectLatest
14 import kotlinx.coroutines.launch
15
16 import com.example.zennfit.network.ApiClient
17 import com.example.zennfit.data.AppDatabase
18 import com.example.zennfit.data.AlatGymRepository
19 import com.example.zennfit.model.getLocalizedDescription
20 import com.example.zennfit.model.getLocalizedName
21 import com.example.zennfit.model.getMainImageUrl
22 import com.example.zennfit.model.getMainVideoUrl
23
24 class HomeFragment : Fragment() {
25
26     private var _binding: FragmentHomeBinding? = null
```

```

private val binding get() = _binding!!

private val viewModel: AlatGymViewModel by viewModels {
    Log.d("HomeFragmentInit", "Starting ViewModel
initialization.")
    try {
        val exerciseInfoDao =
AppDatabase.getDatabase(requireContext()).exerciseInfoDao()
        Log.d("HomeFragmentInit", "ExerciseInfoDao initialized.")
        val wgerApiService = ApiClient.wgerApiService
        Log.d("HomeFragmentInit", "WgerApiService initialized.")
        val repository = AlatGymRepository(wgerApiService,
exerciseInfoDao)
        Log.d("HomeFragmentInit", "AlatGymRepository
initialized.")
        AlatGymViewModelFactory(repository)
    } catch (e: Exception) {
        Log.e("HomeFragmentInit", "Error during ViewModel
initialization: ${e.message}", e)
        throw e
    }
}

override fun onCreateView(
    inflater: LayoutInflater, container: ViewGroup?,
    savedInstanceState: Bundle?
): View {
    _binding = FragmentHomeBinding.inflate(inflater, container,
false)
    return binding.root
}

override fun onViewCreated(view: View, savedInstanceState:
Bundle?) {
    super.onViewCreated(view, savedInstanceState)
    Log.d("HomeFragment", "onViewCreated called.")

    val adapter = AlatGymAdapter(emptyList()) { exerciseInfo ->
        viewModel.onItemClicked(exerciseInfo)
        Log.d("HomeFragment", "Tombol item diklik:
${exerciseInfo.localizedName()}")
    }

    binding.recyclerView.layoutManager =
LinearLayoutManager(requireContext())
    binding.recyclerView.adapter = adapter

    lifecycleScope.launch {
        viewModel.alatList.collectLatest { exerciseInfoList ->
            Log.d("HomeFragment", "Data alat diambil dari
ViewModel: ${exerciseInfoList.size} item")
            Log.d("HomeFragment", "Calling adapter.updateData()
with ${exerciseInfoList.size} items.")
            adapter.updateData(exerciseInfoList)
        }
    }
}

```

```

        lifecycleScope.launch {
            viewModel.selectedItem.collectLatest { selected ->
                selected?.let { exerciseInfo ->
                    val fragment = DetailFragment().apply {
                        arguments = Bundle().apply {
                            putString("nama",
exerciseInfo.getLocalizedName())
                            putString("deskripsi",
exerciseInfo.getLocalizedDescription())
                            putString("imageUrl",
exerciseInfo.getMainImageUrl())
                            putString("videoUrl",
exerciseInfo.getMainVideoUrl())
                        }
                    }

                    parentFragmentManager.beginTransaction()
                        .replace(R.id.fragment_container, fragment)
                        .addToBackStack(null)
                        .commit()

                    viewModel.clearSelectedItem()
                    Log.d("HomeFragment", "Berpindah ke
DetailFragment: ${exerciseInfo.getLocalizedName()}")
                }
            }
        }

        override fun onDestroyView() {
            super.onDestroyView()
            _binding = null
        }
    }
}

```

Table 1. Source Code Jawaban Soal 1

2. DetailFragment.kt

```

1 package com.example.zennfit
2
3 import android.os.Bundle
4 import android.view.LayoutInflater
5 import android.view.View
6 import android.view.ViewGroup
7 import androidx.fragment.app.Fragment
8 import com.bumptech.glide.Glide
9 import com.example.zennfit.databinding.FragmentDetailBinding
10 import android.util.Log
11
12 import com.google.android.exoplayer2.ExoPlayer
13 import com.google.android.exoplayer2.MediaItem
14
15 class DetailFragment : Fragment() {
16
17     private var _binding: FragmentDetailBinding? = null
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78
79
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100

```



```

private val binding get() = _binding!!

private var player: ExoPlayer? = null
private var playWhenReady = true
private var playbackPosition = 0L

companion object {
    fun newInstance(nama: String, deskripsi: String, imageUrl:
String?, videoUrl: String?): DetailFragment {
        val fragment = DetailFragment()
        val bundle = Bundle().apply {
            putString("nama", nama)
            putString("deskripsi", deskripsi)
            putString("imageUrl", imageUrl)
            putString("videoUrl", videoUrl)
        }
        fragment.arguments = bundle
        return fragment
    }
}

override fun onCreateView(
    inflater: LayoutInflater, container: ViewGroup?,
    savedInstanceState: Bundle?
): View {
    _binding = FragmentDetailBinding.inflate(inflater, container,
false)
    return binding.root
}

override fun onViewCreated(view: View, savedInstanceState:
Bundle?) {
    super.onViewCreated(view, savedInstanceState)

    val nama = arguments?.getString("nama")
    val deskripsi = arguments?.getString("deskripsi") ?:
"Deskripsi tidak tersedia."
    val imageUrl = arguments?.getString("imageUrl")
    val videoUrl = arguments?.getString("videoUrl")

    Log.d("DetailFragment", "Menampilkan detail untuk: $nama")
    Log.d("DetailFragment", "Deskripsi: $deskripsi")
    Log.d("DetailFragment", "Image URL: $imageUrl")
    Log.d("DetailFragment", "Video URL: $videoUrl")

    activity?.title = nama
    binding.namaAlat.text = nama
    binding.deskripsiText.text = deskripsi

    if (!imageUrl.isNullOrEmpty()) {
        binding.alatImage2.visibility = View.VISIBLE
        Glide.with(this)
            .load(imageUrl)
            .placeholder(R.drawable.placeholder_image)
            .error(R.drawable.placeholder_image)
            .into(binding.alatImage2)
    } else {

```

```

        binding.alatImage2.visibility = View.GONE
    }

    if (!videoUrl.isNullOrEmpty()) {
        binding.videoPlayerView.visibility = View.VISIBLE
        initializePlayer(videoUrl)
    } else {
        binding.videoPlayerView.visibility = View.GONE
        Log.d("DetailFragment", "No video URL provided,
PlayerView hidden.")
    }
}

private fun initializePlayer(videoUrl: String) {
    if (player == null) {
        player = ExoPlayer.Builder(requireContext())
            .build()
        binding.videoPlayerView.player = player
    }

    val mediaItem = MediaItem.fromUri(videoUrl)
    player?.setMediaItem(mediaItem)
    player?.seekTo(playbackPosition)
    player?.playWhenReady = playWhenReady
    player?.prepare()
    player?.repeatMode = ExoPlayer.REPEAT_MODE_ALL
    Log.d("DetailFragment", "ExoPlayer initialized and trying to
play: $videoUrl")
}

override fun onStart() {
    super.onStart()
    val videoUrl = arguments?.getString("videoUrl")
    if (!videoUrl.isNullOrEmpty() && player == null) {
        initializePlayer(videoUrl)
    }
}

override fun onResume() {
    super.onResume()
    player?.playWhenReady = true
    Log.d("DetailFragment", "ExoPlayer onResume: playWhenReady =
true")
}

override fun onPause() {
    super.onPause()
    player?.playWhenReady = false

    playbackPosition = player?.currentPosition ?: 0L
    playWhenReady = player?.playWhenReady ?: true

    Log.d("DetailFragment", "ExoPlayer onPause: playWhenReady =
$playWhenReady, position = $playbackPosition")
}

override fun onStop() {

```

	<pre> super.onStop() releasePlayer() Log.d("DetailFragment", "ExoPlayer onStop: Player released.") } private fun releasePlayer() { if (player != null) { player?.release() player = null Log.d("DetailFragment", "ExoPlayer resources released.") } } override fun onDestroyView() { super.onDestroyView() _binding = null } } </pre>
--	---

Table 2. Source Code Jawaban Soal 1

3. AlatGymViewModel

1	package com.example.zennfit
2	
3	import androidx.lifecycle.ViewModel
4	import androidx.lifecycle.viewModelScope
5	import com.example.zennfit.data.AlatGymRepository
6	import com.example.zennfit.model.ExerciseInfo
7	import com.example.zennfit.model.getLocalizedName
8	import com.example.zennfit.model.getLocalizedDescription
9	import com.example.zennfit.model.getMainImageUrl
10	import com.example.zennfit.model.getMainVideoUrl
11	import kotlinx.coroutines.flow.MutableStateFlow
	import kotlinx.coroutines.flow.StateFlow
	import kotlinx.coroutines.flow.asStateFlow
	import kotlinx.coroutines.launch
	import android.util.Log
	class AlatGymViewModel(
	private val repository: AlatGymRepository
	) : ViewModel() {
	private val _alatList =
	MutableStateFlow<List<ExerciseInfo>>(emptyList())
	val alatList: StateFlow<List<ExerciseInfo>> =
	_alatList.asStateFlow()
	private val _selectedItem = MutableStateFlow<ExerciseInfo?>(null)
	val selectedItem: StateFlow<ExerciseInfo?> =
	_selectedItem.asStateFlow()
	private val curatedExerciseIds = listOf(
	73, // Bench Press
	272, // Hammercurls

```

        465,
        567,
        194,
        475
    )

    init {
        Log.d("AlatGymViewModel", "ViewModel init block executed.")
        loadCuratedExercises()
    }

    private fun loadCuratedExercises() {
        Log.d("AlatGymViewModel", "loadCuratedExercises() called to
fetch specific IDs.")
        viewModelScope.launch {
            val loadedExercises = mutableListOf<ExerciseInfo>()
            for (id in curatedExerciseIds) {
                Log.d("AlatGymViewModel", "Attempting to fetch
exercise with ID: $id")
                val exercise = repository.getExerciseInfoById(id)
                if (exercise != null) {
                    val hasImage =
!exercise.getMainImageUrl().isNullOrEmpty()
                    val hasDescription =
!(exercise.getLocalizedDescription() == "Deskripsi tidak tersedia."
|| exercise.getLocalizedDescription().isBlank())
                    val hasVideo =
!exercise.getMainVideoUrl().isNullOrEmpty()

                    if (hasImage && hasDescription) {
                        loadedExercises.add(exercise)
                        Log.d("AlatGymViewModel", "Successfully added
complete exercise ID $id: ${exercise.getLocalizedName()} | HasImg:
$hasImage | HasDesc: $hasDescription | HasVid: $hasVideo")
                    } else {
                        Log.w("AlatGymViewModel", "Exercise ID $id
(${exercise.getLocalizedName()}) is not 'complete' enough (missing
img/desc). Not adding to list. HasImg: $hasImage | HasDesc:
$hasDescription | HasVid: $hasVideo")
                    }

                    } else {
                        Log.w("AlatGymViewModel", "Failed to fetch
exercise with ID: $id. Not adding to list.")
                    }
                }
            _alatList.value = loadedExercises
            Log.d("AlatGymViewModel", "Finished loading curated
exercises. Displaying ${loadedExercises.size} items.")
        }

        fun onItemClick(item: ExerciseInfo) {
            _selectedItem.value = item
            Log.d("AlatGymViewModel", "Item diklik:
${item.getLocalizedName()}")
        }
    }

```

	<pre> fun clearSelectedItem() { _selectedItem.value = null Log.d("AlatGymViewModel", "Selected item cleared.") } fun refreshData() { Log.d("AlatGymViewModel", "refreshData() called. Reloading curated exercises.") loadCuratedExercises() } } </pre>
--	---

Table 3. Source Code Jawaban Soal 1

4. AlatGymViewModelFactory

1	package com.example.zennfit
2	
3	import androidx.lifecycle.ViewModel
4	import androidx.lifecycle.ViewModelProvider
5	import com.example.zennfit.data.AlatGymRepository
6	
7	class AlatGymViewModelFactory(
8	private val repository: AlatGymRepository
9	) : ViewModelProvider.Factory {
10	override fun <T : ViewModel> create(modelClass: Class<T>): T {
11	if
	(modelClass.isAssignableFrom(AlatGymViewModel::class.java)) {
	@Suppress("UNCHECKED_CAST")
	return AlatGymViewModel(repository) as T
	}
	throw IllegalArgumentException("Unknown ViewModel class")
	}
	}

Table 4. Source Code Jawaban Soal 1

5. WgerApiService

1	package com.example.zennfit.network
2	
3	import com.example.zennfit.model.WgerExerciseInfoResponse
4	import com.example.zennfit.model.ExerciseInfo
5	import retrofit2.Response
6	import retrofit2.http.GET
7	import retrofit2.http.Path
8	import retrofit2.http.Query
9	
10	interface WgerApiService {
11	@GET("api/v2/exerciseinfo/")
	suspend fun getExerciseInfo(
	@Query("search") query: String? = null
	): Response<WgerExerciseInfoResponse>

	<pre> @GET("api/v2/exerciseinfo/{id}/") suspend fun getExerciseInfoById(@Path("id") id: Int): Response<ExerciseInfo> } </pre>
--	---

Table 5. Source Code Jawaban Soal 1

6. ApiClient

1	package com.example.zennfit.network
2	
3	import okhttp3.OkHttpClient
4	import okhttp3.logging.HttpLoggingInterceptor
5	import retrofit2.Retrofit
6	import retrofit2.converter.gson.GsonConverterFactory
7	import java.util.concurrent.TimeUnit
8	object ApiClient {
9	
10	private const val BASE_URL = "https://wger.de/"
11	
	private val loggingInterceptor = HttpLoggingInterceptor().apply {
	level = HttpLoggingInterceptor.Level.BODY
	}
	private val okHttpClient = OkHttpClient.Builder()
	.addInterceptor(loggingInterceptor)
	.connectTimeout(30, TimeUnit.SECONDS)
	.readTimeout(30, TimeUnit.SECONDS)
	.writeTimeout(30, TimeUnit.SECONDS)
	.build()
	private val retrofit: Retrofit by lazy {
	Retrofit.Builder()
	.baseUrl(BASE_URL)
	.client(okHttpClient)
	.addConverterFactory(GsonConverterFactory.create())
	.build()
	}
	val wgerApiService: WgerApiService by lazy {
	retrofit.create(WgerApiService::class.java)
	}
	}

Table 6. Source Code Jawaban Soal 1

7. WgerEquipmentInfoResponse

1	package com.example.zennfit.model
2	
3	import com.google.gson.annotations.SerializedName
4	
5	data class WgerExerciseInfoResponse(

6	@SerializedName("count") val count: Int,
7	@SerializedName("next") val next: String?,
8	@SerializedName("previous") val previous: String?,
9	@SerializedName("results") val results: List<ExerciseInfo> //
10	Ganti List<Equipment> ke List<ExerciseInfo>
11	)

Table 7. Source Code Jawaban Soal 1

8. ExerciseInfo

1	package com.example.zennfit.model
2	
3	import com.google.gson.annotations.SerializedName
4	import android.text.Html
5	import android.os.Build
6	data class ExerciseInfo(
7	@SerializedName("id") val id: Int,
8	@SerializedName("uuid") val uuid: String,
9	@SerializedName("created") val created: String?,
10	@SerializedName("last_update") val lastUpdate: String?,
11	@SerializedName("last_update_global") val lastUpdateGlobal:
	String?,
	@SerializedName("category") val category: Category?,
	@SerializedName("muscles") val muscles: List<Muscle>?,
	@SerializedName("muscles_secondary") val musclesSecondary:
	List<Muscle>?,
	@SerializedName("equipment") val equipment:
	List<EquipmentDetail>?,
	@SerializedName("license") val license: License?,
	@SerializedName("license_author") val licenseAuthor: String?,
	@SerializedName("images") val images: List<ExerciseImage>?,
	@SerializedName("translations") val translations:
	List<ExerciseTranslation>?,
	@SerializedName("variations") val variations: Any?,
	@SerializedName("videos") val videos: List<ExerciseVideo>?,
	@SerializedName("author_history") val authorHistory:
	List<String>?,
	@SerializedName("total_authors_history") val totalAuthorsHistory:
	List<String>?
	)
	data class Category(
	@SerializedName("id") val id: Int,
	@SerializedName("name") val name: String
	)
	data class Muscle(
	@SerializedName("id") val id: Int,
	@SerializedName("name") val name: String,
	@SerializedName("name_en") val nameEn: String?,
	@SerializedName("is_front") val isFront: Boolean?,
	@SerializedName("image_url_main") val imageUrlMain: String?,
	@SerializedName("image_url_secondary") val imageUrlSecondary:

```

String?
)

data class EquipmentDetail(
    @SerializedName("id") val id: Int,
    @SerializedName("name") val name: String
)

data class License(
    @SerializedName("id") val id: Int,
    @SerializedName("full_name") val fullName: String,
    @SerializedName("short_name") val shortName: String,
    @SerializedName("url") val url: String
)

data class ExerciseImage(
    @SerializedName("id") val id: Int,
    @SerializedName("exercise_base") val exerciseBaseId: Int?,
    @SerializedName("image") val imageUrl: String,
    @SerializedName("is_main") val isMain: Boolean,
    @SerializedName("uuid") val uuid: String
)

data class ExerciseVideo(
    @SerializedName("id") val id: Int,
    @SerializedName("exercise_base") val exerciseBaseId: Int?,
    @SerializedName("video") val videoUrl: String,
    @SerializedName("is_main") val isMain: Boolean,
    @SerializedName("uuid") val uuid: String
)

data class ExerciseTranslation(
    @SerializedName("id") val id: Int,
    @SerializedName("uuid") val uuid: String,
    @SerializedName("name") val name: String,
    @SerializedName("exercise") val exerciseId: Int?,
    @SerializedName("description") val description: String,
    @SerializedName("created") val created: String?,
    @SerializedName("language") val languageId: Int,
    @SerializedName("aliases") val aliases: List<Any>?,
    @SerializedName("notes") val notes: List<Any>?,
    @SerializedName("license") val license: Int?,
    @SerializedName("license_title") val licenseTitle: String?,
    @SerializedName("license_object_url") val licenseObjectUrl:
String?,
    @SerializedName("license_author") val licenseAuthor: String?,
    @SerializedName("license_author_url") val licenseAuthorUrl:
String?,
    @SerializedName("license_derivative_source_url") val
licenseDerivativeSourceUrl: String?,
    @SerializedName("author_history") val authorHistory:
List<String>?
)

fun ExerciseInfo.getLocalizedName(languageId: Int = 2): String {
    val localizedTranslation = translations?.firstOrNull {

```


	<pre> it.languageId == languageId } return localizedTranslation?.name ?: translations?.firstOrNull()?.name ?: "Nama tidak tersedia" } fun ExerciseInfo.getLocalizedDescription(languageId: Int = 2): String { val localizedTranslation = translations?.firstOrNull { it.languageId == languageId } val htmlDescription = localizedTranslation?.description ?: translations?.firstOrNull()?.description return if (!htmlDescription.isNullOrEmpty()) { if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.N) { Html.fromHtml(htmlDescription, Html.FROM_HTML_MODE_COMPACT).toString() } else { @Suppress("DEPRECATION") Html.fromHtml(htmlDescription).toString() } } else { "Deskripsi tidak tersedia." } } fun ExerciseInfo.getMainImageUrl(): String? { return images?.firstOrNull { it.isMain }?.imageUrl ?: images?.firstOrNull()?.imageUrl } fun ExerciseInfo.getMainVideoUrl(): String? { return videos?.firstOrNull { it.isMain }?.videoUrl ?: videos?.firstOrNull()?.videoUrl } </pre>
--	---

Table 8. Source Code Jawaban Soal 1

9. ExerciseInfoEntity

1	package com.example.zennfit.data
2	
3	import androidx.room.Entity
4	import androidx.room.PrimaryKey
5	
6	@Entity(tableName = "exercise_info_table")
7	data class ExerciseInfoEntity(
8	@PrimaryKey val id: Int,
9	val name: String,
10	val description: String,
11	val imageUrl: String?,
	val videoUrl: String?
	)

Table 9. Source Code Jawaban Soal 1

10. ExerciseInfoDao

```
1 package com.example.zennfit.data
2
3 import androidx.room.Dao
4 import androidx.room.Insert
5 import androidx.room.OnConflictStrategy
6 import androidx.room.Query
7 import kotlinx.coroutines.flow.Flow
8
9 @Dao
10 interface ExerciseInfoDao {
11     @Query("SELECT * FROM exercise_info_table ORDER BY name ASC")
12     fun getAllExerciseInfo(): Flow<List<ExerciseInfoEntity>>
13
14     @Insert(onConflict = OnConflictStrategy.REPLACE)
15     suspend fun insertAll(exerciseInfo: List<ExerciseInfoEntity>)
16
17     @Query("DELETE FROM exercise_info_table")
18     suspend fun deleteAllExerciseInfo()
19 }
```

Table 10. Source Code Jawaban Soal 1

11. AppDataBase

```
1 package com.example.zennfit.data
2
3 import android.content.Context
4 import androidx.room.Database
5 import androidx.room.Room
6 import androidx.room.RoomDatabase
7
8 @Database(entities = [ExerciseInfoEntity::class], version = 1,
9 exportSchema = false)
10 abstract class AppDatabase : RoomDatabase() {
11     abstract fun exerciseInfoDao(): ExerciseInfoDao
12
13     companion object {
14         @Volatile
15         private var INSTANCE: AppDatabase? = null
16
17         fun getDatabase(context: Context): AppDatabase {
18             return INSTANCE ?: synchronized(this) {
19                 val instance = Room.databaseBuilder(
20                     context.applicationContext,
21                     AppDatabase::class.java,
22                     "zennfit_database"
23                 ).build()
24                 INSTANCE = instance
25                 instance
26             }
27         }
28     }
29 }
```

	<pre> } } }</pre>
--	-------------------------------

Table 11. Source Code Jawaban Soal 1

12. AlatGymRepository

```

1 package com.example.zennfit.data
2
3 import android.util.Log
4 import com.example.zennfit.model.*
5 import com.example.zennfit.network.WgerApiService
6 import kotlinx.coroutines.Dispatchers
7 import kotlinx.coroutines.flow.Flow
8 import kotlinx.coroutines.flow.flowOn
9 import kotlinx.coroutines.flow.map
10 import kotlinx.coroutines.flow.onEach
11 import kotlinx.coroutines.flow.catch
12 import kotlinx.coroutines.coroutineScope
13 import kotlinx.coroutines.withContext
14
15 class AlatGymRepository(
16     private val wgerApiService: WgerApiService,
17     private val exerciseInfoDao: ExerciseInfoDao
18 ) {
19     fun getExerciseInfo(searchQuery: String? = null):
20     Flow<List<ExerciseInfo>> = exerciseInfoDao.getAllExerciseInfo()
21         .map { entities ->
22             Log.d("AlatGymRepositoryFlow", "Mapping ${entities.size}
23 items from Room entities to external models.")
24             entities.map { it.toExternalModel() }
25         }
26         .onEach {
27             Log.d("AlatGymRepositoryFlow", "Flow emitted data to
28 ViewModel (from Room): ${it.size} items.")
29
30             if (it.isEmpty() || searchQuery != null) {
31                 Log.d("AlatGymRepositoryFlow", "Attempting to fetch
32 from network with query: '${searchQuery ?: "none"}'")
33                 fetchAndSaveToRoom(searchQuery)
34             } else {
35                 Log.d("AlatGymRepositoryFlow", "Room contains data,
36 skipping immediate network fetch for now.")
37             }
38         }
39         .catch { e ->
40             Log.e("AlatGymRepositoryFlow", "Error in flow collection:
41 ${e.message}", e)
42             emit(emptyList())
43         }
44         .flowOn(Dispatchers.IO)
45
46     private suspend fun fetchAndSaveToRoom(query: String?) {
47         coroutineScope {
48             try {

```

```

        Log.d("AlatGymRepositoryNetwork", "Starting network
fetch with query: '${query ?: "none"}'")
        val response = wgerApiService.getExerciseInfo(query)
        Log.d("AlatGymRepositoryNetwork", "Network response
received. isSuccessful: ${response.isSuccessful}, Code:
${response.code()}, Message: ${response.message()}")

        if (response.isSuccessful) {
            val apiResponse = response.body()
            if (apiResponse != null) {
                val exerciseInfoList = apiResponse.results
                if (exerciseInfoList != null &&
exerciseInfoList.isNotEmpty()) {
                    Log.d("AlatGymRepositoryNetwork", "API
response body is not null. Results size: ${exerciseInfoList.size}")

                    val exerciseInfoEntities =
exerciseInfoList.map { it.toEntity() }

                    exerciseInfoDao.deleteAllExerciseInfo()

exerciseInfoDao.insertAll(exerciseInfoEntities)
                    Log.d("AlatGymRepositoryNetwork",
"Successfully fetched ${exerciseInfoList.size} items from network and
saved to Room.")

                } else {
                    Log.w("AlatGymRepositoryNetwork", "API
response results are null or empty for query: '${query ?: "none"}'")
                    if (query != null) {

exerciseInfoDao.deleteAllExerciseInfo()
                        Log.d("AlatGymRepositoryNetwork",
"Cleared Room data as network search for '${query}' returned empty.")
                    }
                }
            } else {
                Log.w("AlatGymRepositoryNetwork", "API
response body is null.")
            }
        } else {
            Log.e("AlatGymRepositoryNetwork", "API call
failed with code: ${response.code()}, message: ${response.message()},
Raw Error Body: ${response.errorBody()?.string()}")
        }
    } catch (e: Exception) {
        Log.e("AlatGymRepositoryNetwork", "Error during
network fetch for query '${query ?: "none"}': ${e.message}", e)
    }
}

suspend fun getExerciseInfoById(id: Int): ExerciseInfo? {
    return withContext(Dispatchers.IO) {
        try {
            Log.d("AlatGymRepositoryNetwork", "Attempting to fetch
exercise with ID: $id")

```

```

        val response = wgerApiService.getExerciseInfoById(id)
        Log.d("AlatGymRepositoryById", "Response for ID $id:
isSuccessful: ${response.isSuccessful}, Code: ${response.code()},
Message: ${response.message()}")

        if (response.isSuccessful) {
            val exercise = response.body()
            if (exercise != null) {
                Log.d("AlatGymRepositoryById", "Successfully
fetched exercise ID $id: ${exercise.localizedName}")
                exercise
            } else {
                Log.w("AlatGymRepositoryById", "API response
body for ID $id is null.")
                null
            }
        } else {
            Log.e("AlatGymRepositoryById", "API call failed
for ID $id with code: ${response.code()}, message:
${response.message()}, Raw Error Body:
${response.errorBody()?.string()}")
            null
        }
    } catch (e: Exception) {
        Log.e("AlatGymRepositoryById", "Error fetching
exercise with ID $id: ${e.message}", e)
        null
    }
}

}

private fun ExerciseInfo.toEntity(): ExerciseInfoEntity {
    return ExerciseInfoEntity(
        id = this.id,
        name = this.localizedName(),
        description = this.localizedDescription(),
        imageUrl = this.mainImageUrl(),
        videoUrl = this.mainVideoUrl()
    )
}

private fun ExerciseInfoEntity.toExternalModel(): ExerciseInfo {
    return ExerciseInfo(
        id = this.id,
        uuid = "",
        created = null,
        lastUpdate = null,
        lastUpdateGlobal = null,
        category = null,
        muscles = emptyList(),
        musclesSecondary = emptyList(),
        equipment = emptyList(),
        license = null,
        licenseAuthor = null,
        images = if (this.imageUrl != null)
listOf(ExerciseImage(0, null, this.imageUrl, true, "")) else null,
        videos = if (this.videoUrl != null)

```

```

listOf(ExerciseVideo(0, null, this.videoUrl, true, "")) else null,
    translations = listOf(
        ExerciseTranslation(
            id = 0,
            uuid = "",
            name = this.name,
            exerciseId = this.id,
            description = this.description,
            created = null,
            languageId = 2,
            aliases = emptyList(),
            notes = emptyList(),
            license = null,
            licenseTitle = null,
            licenseObjectUrl = null,
            licenseAuthor = null,
            licenseAuthorUrl = null,
            licenseDerivativeSourceUrl = null,
            authorHistory = emptyList()
        )
    ),
    variations = null,
    authorHistory = emptyList(),
    totalAuthorsHistory = emptyList()
)
}

```

Table 12. Source Code Jawaban Soal 1

13. AlatGymAdapter

```

1 package com.example.zennfit
2
3 import android.view.LayoutInflater
4 import android.view.ViewGroup
5 import android.content.Intent
6 import android.net.Uri
7 import android.view.View
8 import androidx.recyclerview.widget.RecyclerView
9 import com.example.zennfit.databinding.ListItemBinding
10 import com.example.zennfit.model.ExerciseInfo
11 import com.bumptech.glide.Glide
12 import com.example.zennfit.model.getLocalizedDescription
13 import com.example.zennfit.model.getLocalizedName
14 import com.example.zennfit.model.getMainImageUrl
15 import android.util.Log
16
17 class AlatGymAdapter(
18     private var alatList: List<ExerciseInfo>,
19     private val onClick: (ExerciseInfo) -> Unit
20 ) : RecyclerView.Adapter<AlatGymAdapter.ViewHolder>() {
21
22     class ViewHolder(val binding: ListItemBinding) :
23         RecyclerView.ViewHolder(binding.root)
24
25 }

```

```

        override fun onCreateViewHolder(parent: ViewGroup, viewType:
Int): ViewHolder {
            val binding =
ListItemBinding.inflate(LayoutInflater.from(parent.context), parent,
false)
            return ViewHolder(binding)
        }

        override fun onBindViewHolder(holder: ViewHolder, position: Int)
{
            val alat = alatList[position]

            holder.binding.namaAlat.text = alat.getLocalizedName()

            val description = alat.getLocalizedDescription()
            if (description == "Deskripsi tidak tersedia." ||
description.isBlank()) {
                holder.binding.descDetail.visibility = View.GONE
            } else {
                holder.binding.descDetail.visibility = View.VISIBLE
                holder.binding.descDetail.text = description
            }

            val imageUrl = alat.getMainImageUrl()
            if (!imageUrl.isNullOrEmpty()) {
                holder.binding.alatImageList.visibility = View.VISIBLE
                Glide.with(holder.itemView.context)
                    .load(imageUrl)
                    .placeholder(R.drawable.placeholder_image)
                    .error(R.drawable.placeholder_image)
                    .into(holder.binding.alatImageList)
            } else {
                holder.binding.alatImageList.visibility = View.GONE
            }

            holder.binding.viewButton.setOnClickListener {
                onClick(alat)
            }

            holder.binding.browserButton.setOnClickListener {
                val context = holder.itemView.context
                val wgerUrl =
"https://wger.de/id/exercise/${alat.id}/view"

                Log.d("BrowserUrlDebug", "Final URL sent to browser:
$wgerUrl")

                val intent = Intent(
                    Intent.ACTION_VIEW,
                    Uri.parse(wgerUrl)
                )

                try {
                    context.startActivity(intent)
                    Log.d("BrowserUrlDebug", "URL opened successfully.")
                } catch (e: Exception) {
                    Log.e("BrowserUrlDebug", "Could not open URL:

```

```

$swaggerUrl. Error: ${e.message}")
    }
}

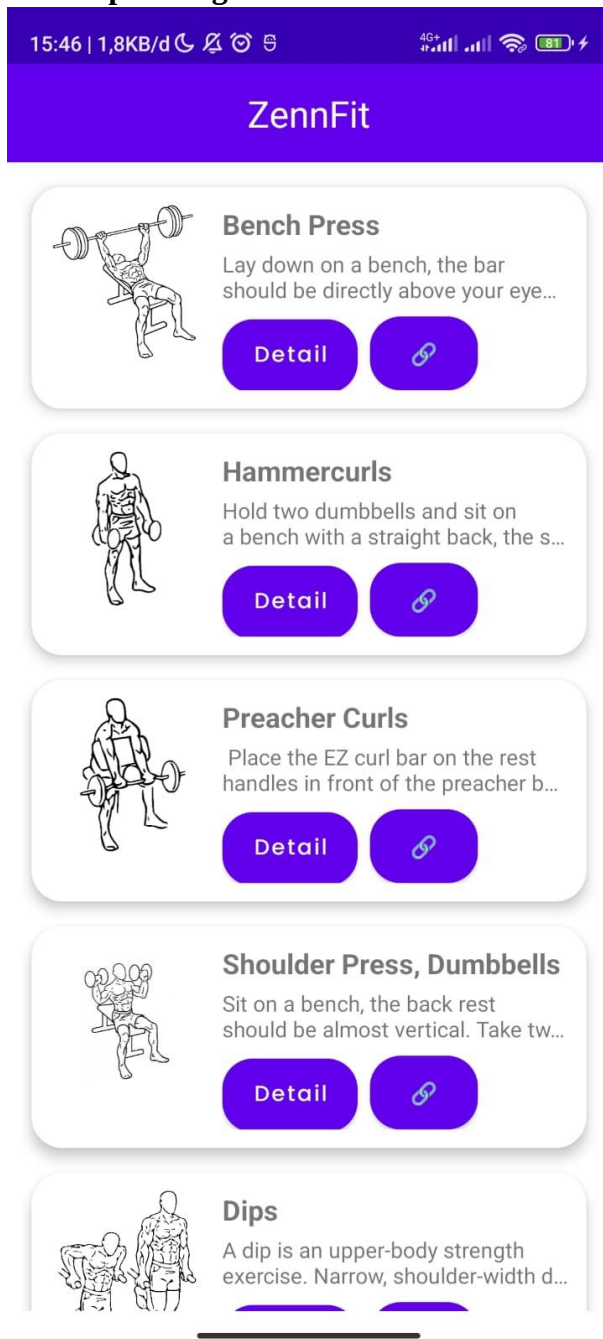
override fun getItemCount(): Int {
    val count = alatList.size
    Log.d("AlatGymAdapter", "getItemCount() called. Returning:
$count")
    return count
}

fun updateData(newList: List<ExerciseInfo>) {
    Log.d("AlatGymAdapter", "updateData() called with
${newList.size} items.")
    alatList = newList
    notifyDataSetChanged()
    Log.d("AlatGymAdapter", "notifyDataSetChanged() called.")
}
}

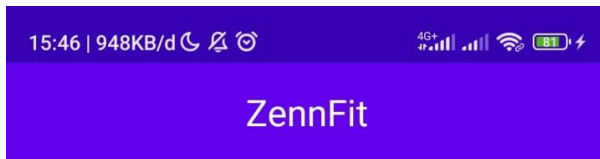
```

Table 13. Source Code Jawaban Soal 1

B. Output Program



Gambar 1. Screenshot hasil soal 1



Bench Press

Lay down on a bench, the bar should be directly above your eyes, the knees are somewhat angled and the feet are firmly on the floor. Concentrate, breath deeply and grab the bar more than shoulder wide. Bring it slowly down till it briefly touches your chest at the height of your nipples. Push the bar up.

If you train with a high weight it is advisable to have a spotter that can help you up if you can't lift the weight on your own.

With the width of the grip you can also control which part of the chest is trained more:

wide grip: outer chest muscles

narrow grip: inner chest muscles and triceps



Gambar 2. Screenshot hasil soal 1

C. Pembahasan

1. HomeFragment

Langsung saja ke onCreateView() pada kode `_binding = FragmentHomeBinding.inflate(inflater, container, false)` ini melakukan inflasi (mengubah XML menjadi View object), kemudian menyimpan View tersebut ke `_binding`. Lalu onViewCreated, isinya menginisialisasi RecyclerView.Adapter untuk

daftar alat gym. Digunakan ketika user klik salah satu item maka akan memicu `viewModel.onItemClicked()` dan mencetak nama alat yang diklik ke Log. Pada `recyclerView` setup itu mengatur `RecyclerView` agar menampilkan daftar alat secara vertikal. lalu kode ini `viewModel.alatList.collectLatest { ... }` mengamati perubahan pada data list alat dari `ViewModel` dan memperbarui adapter ketika ada data baru. `viewModel.selectedItem.collectLatest { selected -> ... }` ini untuk navigasi ke bagian detail, setelah navigasi, item yang dipilih akan di-clear dari `ViewModel` dengan `viewModel.clearSelectedItem()`. Terakhir `_binding = null` ini membersihkan `_binding` untuk menghindari memory leaks saat view dihancurkan (misalnya saat berpindah fragment).

2. DetailFragment

`class DetailFragment : Fragment()` { ini adalah subclass dari `Fragment` yang digunakan untuk menampilkan detail dari alat gym terpilih, termasuk nama, deskripsi, gambar, dan video. pada View Binding, Menggunakan View Binding (`FragmentDetailBinding`) untuk mengakses elemen XML `fragment_detail.xml`. `_binding` digunakan sebagai penampung sementara; `binding` digunakan untuk akses aman. pada bagian `ExoPlayer`, ini berisi player: objek `ExoPlayer` untuk memutar video. `playWhenReady`: status apakah video langsung diputar setelah `prepare()`. `playbackPosition`: menyimpan posisi terakhir saat video dijeda atau dihentikan. Selanjutnya fungsi `newInstance`, yaitu membuat fragment dengan membawa data (nama, deskripsi, URL gambar, dan URL video) sebagai argumen `Bundle` dan digunakan saat berpindah dari `HomeFragment`. Lalu `onCreateView` dimana ini meng-inflate layout XML `fragment_detail.xml` menggunakan `ViewBinding`. Pada bagian `onViewCreated` mengambil argumen lalu dikirm melalui `newInstance` lalu menampilkan data tadi ke UI dengan `binding`. Selanjutnya menampilkan gambar dengan kondisional, Jika `imageUrl` ada, tampilkan gambar menggunakan library `Glide`. Bila tidak ada gambar, sembunyikan `ImageView`. Lalu menampilkan video juga jika `videoUrl` tersedia, tampilkan `PlayerView` dan inisialisasi `ExoPlayer`. Lalu menginisialisasi `ExoPlayer` yaitu membuat instance `ExoPlayer` dan mempersiapkan media dari URL video lalu atur video agar bisa autoplay (`playWhenReady`) dan mengulang terus (`REPEAT_MODE_ALL`). lalu ke `onStart` dimana jika player belum ada dan URL video tersedia, player diinisialisasi ulang. `onResume` ini melanjutkan playback saat fragment aktif kembali. `onPause` ini simpan posisi video dan status saat fragment tidak aktif. `onStop` dan `releasePlayer` bagian ini akan melepaskan semua resource player saat fragment tidak lagi tampil, mencegah memory leak. Terakhir `onDestroyView` membersihkan `binding` untuk mencegah memory leak.

3. AlatGymViewModel

Pertama deklarasi viewModel, repository: dependensi untuk mengambil data dari sumber (bisa API/local DB). lalu masuk ke state list dan item terpilih yang isinya _alatList: state privat yang menyimpan list alat gym. alatList: versi publik (immutable) untuk di-observe dari UI. _selectedItem: item yang sedang dipilih. selectedItem: expose agar hanya bisa dibaca. Lalu ada daftar ID, ID ini adalah ID dari latihan tertentu yang ingin ditampilkan saja (terkurasi). Mewakili latihan penting seperti Bench Press, Hammer Curls, dll. Lalu ini blok dipanggil saat ViewModel pertama kali dibuat lalu memanggil fungsi loadCuratedExercises() untuk mulai load data. Masuk ke fungsi loadCuratedExercises, Coroutine diluncurkan menggunakan viewModelScope.launch (untuk operasi async) lalu untuk setiap id dalam curatedExerciseIds, ambil data dari repository, cek apakah data memiliki gambar, deskripsi, dan video. Jika data lengkap, tambahkan ke list loadedExercises, jika tidak, lewati. Setelah semua ID dicek, hasil akhir disimpan ke _alatList. Selanjutnya fungsi onItemClick yang menyimpan item yang dipilih pengguna ke selectedItem dan untuk menampilkan detail item di UI. Fungsi clearSelectedItem ini menghapus pilihan pengguna dan dipanggil saat pengguna kembali ke list atau membatalkan aksi.

4. AlatGymViewModelFactory

Pertama mendefinisikan AlatGymViewModelFactory yang mengimplementasi ViewModelProvider.Factory. repository disimpan sebagai properti karena nanti akan diberikan ke ViewModel. override fun <T : ViewModel> create(modelClass: Class<T>): T { method ini dipanggil waktu ViewModelProvider ingin membuat ViewModel. Tipe T adalah tipe ViewModel yang ingin dibuat. Lalu masuk ke kondisional, dicek dulu apakah class yang diminta adalah AlatGymViewModel. kalau ya, buat objek AlatGymViewModel dan kasih parameter repository. as T digunakan untuk meng-cast ke tipe generic T, dan @SuppressWarnings("UNCHECKED_CAST") untuk menenangkan warning. pada kode ini throw IllegalArgumentException("Unknown ViewModel class") jika ternyata class yang diminta bukan AlatGymViewModel, maka dilempar error karena kita nggak tahu cara buat ViewModel lain.

5. AlatGymAdapter

class AlatGymAdapter di dalamnya ada alatList ini untuk daftar data alat gym bertipe ExerciseInfo yang akan ditampilkan. onClick adalah Lambda function yang dijalankan ketika tombol "view" diklik pada item. class ViewHolder(val binding: ListItemBinding) : RecyclerView.ViewHolder(binding.root) viewHolder menyimpan binding dari list_item.xml, agar efisien dalam memetakan data ke view. Lalu di onCreateViewHolder membuat instance ViewHolder dengan memanfaatkan ViewBinding untuk meng-inflate list_item.xml. onBindViewHolder, override fun onBindViewHolder(holder: ViewHolder, position: Int) ini menampilkan data ExerciseInfo ke tampilan yaitu ada nama, deskripsi, gambar, Tombol view yang

ketika tombol "View" diklik, akan men-trigger onClick lambda dari konstruktor, tombol browser yaitu membuat URL untuk membuka halaman detail alat di website wger.de. dan Intent ACTION_VIEW digunakan untuk membuka URL di browser default, jika intent berhasil dibuka, log sukses dan jika gagal (tidak ada browser, dll.), log error. Lalu ada getItemCount yang akan mengembalikan jumlah item yang akan ditampilkan di RecyclerView dan log jumlah data untuk debugging. updateData ini method publik untuk memperbarui data alatList dan memanggil notifyDataSetChanged() agar RecyclerView me-refresh tampilannya.

6. WgerApiService

Langsung ke interface WgerApiService, ini berisi deklarasi fungsi-fungsi untuk mengambil data dari API Wger (Workout Manager Exercise API). Nantinya, Retrofit akan meng-generate implementasi dari interface ini secara otomatis. Selanjutnya Mengambil daftar latihan berdasarkan kata kunci pencarian (query). @GET("api/v2/exerciseinfo/"): melakukan HTTP GET ke endpoint tersebut. @Query("search"): menambahkan parameter pencarian di URL. suspend: karena ini fungsi suspend, maka bisa dipanggil dalam coroutine (viewModelScope.launch). Mengembalikan Response<WgerExerciseInfoResponse>, yaitu respon yang isinya daftar latihan. Selanjutnya Mengambil satu data latihan berdasarkan id. @GET("api/v2/exerciseinfo/{id}/"): endpoint dengan parameter path ({id}). @Path("id"): mengganti {id} di URL dengan nilai id yang diberikan. Return: Response<ExerciseInfo> yang berisi satu objek latihan (ExerciseInfo).

7. ApiClient

private const val BASE_URL = "https://wger.de/" ini adalah URL dasar untuk semua endpoint API. Lalu bagian Logging Interceptor berisi HttpLoggingInterceptor dari OkHttp digunakan untuk melihat log permintaan dan respons HTTP. Level.BODY berarti seluruh isi dari request dan response body akan dicetak di Logcat (penting saat debugging API). OkHttpClient, HTTP client yang digunakan oleh Retrofit untuk menjalankan request. Di sini menambahkan interceptor agar semua aktivitas HTTP bisa dilihat melalui log. Menetapkan batas waktu (timeout) untuk koneksi, membaca, dan menulis selama 30 detik. Selanjutnya ke bagian Retrofit Builder, Retrofit.Builder() digunakan untuk membuat instance Retrofit. baseUrl(BASE_URL): menetapkan URL dasar untuk API. client(okHttpClient): Retrofit menggunakan client dengan konfigurasi timeout dan logging tadi. addConverterFactory(GsonConverterFactory.create()): gunakan Gson untuk mengubah data JSON menjadi objek Kotlin (ExerciseInfo, WgerExerciseInfoResponse, dll). by lazy artinya instance Retrofit hanya akan dibuat satu kali saat pertama kali dipanggil. Terakhir akan menghasilkan service API, membuat instance dari WgerApiService, interface yang didefinisikan untuk endpoint API. retrofit.create(...) akan meng-generate implementasi untuk interface tersebut. Menggunakan by lazy, objek wgerApiService juga hanya akan dibuat saat pertama kali digunakan.

8. WgerEquipmentInfoResponse

Data class `WgerExerciseInfoResponse` ini adalah data class yang digunakan untuk menyimpan data. Otomatis menyediakan `toString()`, `equals()`, `hashCode()`, dan `copy()`. `@SerializedName("count") val count: Int`, mewakili jumlah total data exercise yang tersedia. `@SerializedName("next") val next: String?`, URL halaman berikutnya (untuk pagination) dan bertipe nullable (`String?`) karena bisa null jika tidak ada halaman selanjutnya. `@SerializedName("previous") val previous: String?`, URL halaman sebelumnya (juga untuk pagination) dan bertipe nullable. `@SerializedName("results") val results: List<ExerciseInfo>` ini adalah list dari data latihan (exercise) yang dikembalikan pada halaman tersebut. Tipe datanya adalah `List<ExerciseInfo>`, artinya respons ini akan berisi banyak objek `ExerciseInfo` (data class lain yang mewakili satu item exercise).

9. ExerciseInfo

Data Class Utama: `ExerciseInfo` ini model utama yang mewakili satu data latihan/exercise. Field-nya adalah hasil parsing langsung dari API JSON. Data class `Category(val id: Int, val name: String)` kategori latihan, seperti: Chest, Back, Legs. data class `Muscle` ini menjelaskan otot yang dilatih: `isFront`: apakah otot ada di bagian depan tubuh. `imageUrlMain` dan `imageUrlSecondary`: link gambar otot. data class `EquipmentDetail(val id: Int, val name: String)` nama alat yang digunakan. data class `License` ini informasi lisensi konten seperti: Nama lengkap lisensi (Creative Commons), URL lisensi. data class `ExerciseImage` ini adalah gambar-gambar yang berkaitan dengan latihan, termasuk yang ditandai sebagai gambar utama (`isMain`). data class `ExerciseVideo` ini video latihan (jika tersedia), juga bisa punya status utama (`isMain`). data class `ExerciseTranslation(...)` ini berisi informasi terjemahan seperti nama dan deskripsi latihan dalam berbagai bahasa, ID bahasa (`languageId = 2` untuk Bahasa Indonesia), deskripsi bisa berupa HTML, termasuk info lisensi terjemahan. Pada kode ini fun `ExerciseInfo.getLocalizedName(languageId: Int = 2): String` akan mengambil nama latihan berdasarkan bahasa tertentu (default: 2 = Bahasa Indonesia). Jika tidak tersedia, ambil yang pertama. Jika tetap tidak ada, tampilkan "Nama tidak tersedia". Kode ini fun `ExerciseInfo.getLocalizedDescription(languageId: Int = 2): String` ini sama seperti di atas, tapi untuk deskripsi latihan. Deskripsi biasanya dalam format HTML, jadi diformat ke plain text dengan `Html.fromHtml`. Menyesuaikan versi Android agar kompatibel (`Build.VERSION.SDK_INT`). Output: teks deskripsi latihan dalam bahasa lokal. fun `ExerciseInfo.getMainImageUrl(): String?` ini mengambil URL gambar utama. Jika tidak ada gambar utama (`isMain == true`), ambil gambar pertama jika ada. fun `ExerciseInfo.getMainVideoUrl(): String?` ini juga sama seperti gambar, tetapi untuk video. Mengambil video utama, atau fallback ke video pertama jika `isMain` tidak ada.

10. ExerciseInfoEntity

Pertama deklarasi Entity ExerciseInfoEntity, @Entity(tableName = "exercise_info_table") ini memberitahu Room bahwa class ExerciseInfoEntity ini akan menjadi tabel dalam SQLite database. tableName = "exercise_info_table" adalah nama tabel yang akan digunakan di database. Ini akan dibuat oleh Room secara otomatis saat database dibuat. data class ExerciseInfoEntity ini adalah data class Kotlin yang menyimpan data sederhana dan digunakan untuk mewakili satu baris dari tabel exercise_info_table.

11. ExerciseInfoDao

interface ExerciseInfoDao { ini komponen penting dalam arsitektur Room, bertindak sebagai jembatan antara database dan layer aplikasi lainnya (biasanya repository dan ViewModel). Lalu mengambil semua data dengan FROM exercise_info_table ORDER BY name ASC mengambil semua data dari tabel exercise_info_table, lalu mengurutkannya berdasarkan nama (name) secara ascending (A → Z). Flow<List<ExerciseInfoEntity>>: menggunakan Kotlin Flow agar data bisa berubah secara real-time dan langsung merefleksikan perubahan ke UI. Fungsi ini bersifat non-suspend karena Flow itu sendiri bersifat asynchronous dan dieksekusi secara coroutine-aware. lalu masukkan data dengan @Insert(onConflict = OnConflictStrategy.REPLACE) suspend fun insertAll(exerciseInfo: List<ExerciseInfoEntity>). @Insert ini memberitahu Room bahwa fungsi ini untuk menambahkan data ke tabel. onConflict = OnConflictStrategy.REPLACE jika ada data dengan id yang sama (primary key), maka data lama akan diganti dengan data baru. suspend: Fungsi ini adalah suspend function, yang artinya harus dipanggil dari coroutine atau fungsi suspend lainnya. Ini agar tidak memblok UI thread. exerciseInfo: List<ExerciseInfoEntity> fungsi ini menerima daftar (list) entitas latihan untuk dimasukkan sekaligus. Menghapus semua data dengan @Query("DELETE FROM exercise_info_table") suspend fun deleteAllExerciseInfo(). DELETE FROM exercise_info_table ini menghapus semua data dari tabel exercise_info_table. suspend: Karena proses penghapusan bisa memakan waktu (tergantung ukuran data), fungsi ini harus dijalankan di coroutine agar tidak membebani UI thread.

12. AppDataBase

Pertama ada anotasi database, entities: Menentukan tabel-tabel yang digunakan dalam database ini. Di sini hanya ada 1 entitas yaitu ExerciseInfoEntity. version = 1: Menentukan versi database. Versi ini penting untuk migrasi database jika struktur berubah di masa depan. exportSchema = false: Jika true, Room akan menyimpan file skema (struktur database) untuk dokumentasi atau pengujian. Di sini dimatikan. abstract class AppDatabase : RoomDatabase() { ini adalah RoomDatabase yang digunakan Room untuk membuat dan mengelola database. abstract dimana kelas ini tidak bisa diinstansiasi langsung, tapi akan dibuat oleh Room saat runtime. abstract fun exerciseInfoDao(): ExerciseInfoDao fungsi abstrak ini memberi akses ke DAO

(ExerciseInfoDao), yang menangani query terhadap tabel exercise_info_table. Lalu ada singleton Companion Object, companion object ini digunakan agar fungsi dan properti di dalamnya bersifat statis (seperti static class di Java). INSTANCE untuk menyimpan satu-satunya instance tunggal (singleton) dari AppDatabase. Tujuannya agar database tidak dibuat berulang kali. @Volatile ini untuk memastikan bahwa perubahan nilai INSTANCE dapat terlihat antar thread, penting untuk thread-safety. Lalu Fungsi getDatabase(context: Context), INSTANCE ?: synchronized(this) ini jika INSTANCE belum ada, buat satu dalam blok sinkronisasi agar aman dari kondisi balapan antar-thread. Room.databaseBuilder(...) Room membuat database dengan context.applicationContext untuk menghindari memory leak dari context, AppDatabase::class.java ini kelas database yang ingin dibangun, zennfit_database ini nama file database SQLite yang akan disimpan di memori perangkat. INSTANCE = instance ini untuk menyimpan instance ke dalam variabel static agar tidak dibuat ulang.

13. AlatGymRepository

Didalam class AlatGymRepository ada wgerApiService yaitu objek Retrofit yang digunakan untuk mengambil data dari API dan exerciseInfoDao ini DAO dari Room Database untuk operasi lokal seperti insert, delete, dan get data. fun getExerciseInfo(searchQuery: String? = null): Flow<List<ExerciseInfo>> fungsi ini mengambil data dari database lokal (Room) terlebih dahulu dan hanya mengakses API jika diperlukan. Menggunakan Flow dari Kotlin Coroutines agar data bisa diamati secara real-time dan asynchronous. Fungsi fetchAndSaveToRoom(), private suspend fun fetchAndSaveToRoom(query: String?) fungsi ini mengambil data dari API dan menyimpannya ke database lokal Room. Fungsi getExerciseInfoById(), suspend fun getExerciseInfoById(id: Int): ExerciseInfo? fungsi ini digunakan untuk mengambil satu data latihan berdasarkan ID langsung dari API, bukan dari Room. Lalu dalam Fungsi Konversi Data, toEntity() ini mengubah model dari API (ExerciseInfo) ke bentuk database (ExerciseInfoEntity) mengambil nama, deskripsi, image URL, dan video URL dari model API. toExternalModel() ini untuk mengubah ExerciseInfoEntity dari Room ke model untuk ditampilkan di UI (private fun ExerciseInfoEntity.toExternalModel(): ExerciseInfo), mengisi properti ExerciseInfo dari Room. Karena database hanya menyimpan name, description, imageUrl, dan videoUrl, maka properti lainnya (muscles, equipment, dll) diset null atau emptyList().

D. Tautan Git

Berikut adalah tautan untuk source code yang telah dibuat.

<https://github.com/Specter735/Pemrograman-Mobile.git>